

✓ Data Scripts

These are the scripts I used to collect and augment the training & validation sets.

Setup

```
import pyautogui
import time
import os
from pathlib import Path
import cv2
import numpy as np
import random
import shutil
```

✓ Data Collection

Screen Capture

```
img_number = 1

while True:
    screenshot = pyautogui.screenshot()
    filename = f"z_screen_{img_number}.png"
    screenshot.save(filename)
    print(f"Image saved as {filename}")
    img_number += 1
    time.sleep(5)
```

```
↩ Image saved as z_screen_1.png
Image saved as z_screen_2.png

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[1], line 12
     10 print(f"Image saved as {filename}")
     11 img_number += 1
--> 12 time.sleep(5)

KeyboardInterrupt:
```

✓ Camera Capture

```
camera = cv2.VideoCapture(1)

if not camera.isOpened():
    raise IOError("Cannot open webcam")

img_number = 1

while True:
    ret, frame = camera.read()

    if not ret:
        print("Error: Cannot capture frame.")
        break

    filename = f"z_screen_{img_number}.jpg"

    cv2.imwrite(filename, frame)
    print(f"Image saved as {filename}")

    img_number += 1
    time.sleep(15)

camera.release()
print("Webcam released.")
```

```
➡ Image saved as z_screen_1.jpg
Image saved as z_screen_2.jpg
Image saved as z_screen_3.jpg
Image saved as z_screen_4.jpg
Image saved as z_screen_5.jpg
Image saved as z_screen_6.jpg
Image saved as z_screen_7.jpg
Image saved as z_screen_8.jpg
Image saved as z_screen_9.jpg
Image saved as z_screen_10.jpg
Image saved as z_screen_11.jpg
Image saved as z_screen_12.jpg
Image saved as z_screen_13.jpg
Image saved as z_screen_14.jpg
Image saved as z_screen_15.jpg
Image saved as z_screen_16.jpg
Image saved as z_screen_17.jpg
Image saved as z_screen_18.jpg
Image saved as z_screen_19.jpg
Image saved as z_screen_20.jpg
Image saved as z_screen_21.jpg
Image saved as z_screen_22.jpg
Image saved as z_screen_23.jpg
Image saved as z_screen_24.jpg
Image saved as z_screen_25.jpg
Image saved as z_screen_26.jpg
Image saved as z_screen_27.jpg
Image saved as z_screen_28.jpg
Image saved as z_screen_29.jpg
Image saved as z_screen_30.jpg
Image saved as z_screen_31.jpg
Image saved as z_screen_32.jpg
Image saved as z_screen_33.jpg
Image saved as z_screen_34.jpg
Image saved as z_screen_35.jpg
Image saved as z_screen_36.jpg
Image saved as z_screen_37.jpg
Image saved as z_screen_38.jpg
Image saved as z_screen_39.jpg
Image saved as z_screen_40.jpg
Image saved as z_screen_41.jpg
Image saved as z_screen_42.jpg
```

KeyboardInterrupt Traceback (most recent call last)

```
Cell In[10], line 33
    30     img_number += 1
    32     # Wait for 5 seconds
--> 33     time.sleep(5)
    35 # Release the webcam when the loop is interrupted (e.g., by Ctrl+C)
    36 camera.release()
```

KeyboardInterrupt:

▼ Data Augmentation

```
SOURCE_ROOTS = [
    'data/sol/raw',
    'data/mtga/raw',
    'data/mtgl/raw',
    'data/hs/raw'
]
BASE_OUTPUT_ROOT = Path("data_aug_varsize")
FULL_IMG_ROOT = BASE_OUTPUT_ROOT / "full_images"
CROP_IMG_ROOT = BASE_OUTPUT_ROOT / "cropped_images"
VAL_RATIO = 0.2
CROPS_PER_IMAGE = 6

# Random crop helper
def random_aspect_crop(img, min_crop=0.5, max_crop=0.9):
    h, w, _ = img.shape
    scale = random.uniform(min_crop, max_crop)
    crop_h = int(h * scale)
    crop_w = int(w * scale)

    aspect_jitter = random.uniform(0.85, 1.15)
```

```

crop_w = min(w, int(crop_w * aspect_jitter))
crop_h = min(h, int(crop_h / aspect_jitter))

y0 = random.randint(0, h - crop_h)
x0 = random.randint(0, w - crop_w)
return img[y0:y0+crop_h, x0:x0+crop_w]

#Split original images
print("Splitting original images...")
for src_dir in SOURCE_ROOTS:
    src = Path(src_dir)
    class_name = src.parts[-2]
    images = list(src.glob("*.png")) + \
        list(src.glob("*.jpg")) + \
        list(src.glob("*.jpeg"))
    random.shuffle(images)

    val_count = int(len(images) * VAL_RATIO)
    val_imgs = images[:val_count]
    train_imgs = images[val_count:]

    for split_name, img_list in [("train", train_imgs), ("val", val_imgs)]:
        split_dir = FULL_IMG_ROOT / split_name / class_name
        split_dir.mkdir(parents=True, exist_ok=True)
        for img_path in img_list:
            shutil.copy(img_path, split_dir / img_path.name)

    print(f"[{class_name}] Train: {len(train_imgs)} | Val: {len(val_imgs)}")

# Apply random cropping
print("\nGenerating random crops...")
for split_name in ["train", "val"]:
    for class_dir in (FULL_IMG_ROOT / split_name).glob("*"):
        class_name = class_dir.name
        image_files = list(class_dir.glob("*.png")) + \
            list(class_dir.glob("*.jpg")) + \
            list(class_dir.glob("*.jpeg"))
        output_dir = CROP_IMG_ROOT / split_name / class_name
        output_dir.mkdir(parents=True, exist_ok=True)

        print(f"[{split_name}/{class_name}] Cropping {len(image_files)} images...")

        for img_path in image_files:
            img = cv2.imread(str(img_path))
            if img is None:
                print(f"Skipping unreadable: {img_path}")
                continue

            basename = img_path.stem
            for i in range(CROPS_PER_IMAGE):
                crop = random_aspect_crop(img)
                h, w = crop.shape[:2]
                crop = cv2.resize(crop, (w // 2, h // 2), interpolation=cv2.INTER_AREA)
                out_path = output_dir / f"{basename}_randcrop_{i}.png"
                cv2.imwrite(str(out_path), crop)

print("\nDone!")
print("Full images in: ", FULL_IMG_ROOT)
print("Cropped images in: ", CROP_IMG_ROOT)

```

➡ Splitting original images...

```

[sol] Train: 350 | Val: 87
[mtga] Train: 452 | Val: 112
[mtgl] Train: 273 | Val: 68
[hs] Train: 25 | Val: 6

```

Generating random crops...

```

[train/hs] Cropping 25 images...
[train/mtga] Cropping 452 images...
[train/mtgl] Cropping 273 images...
[train/sol] Cropping 350 images...
[val/hs] Cropping 6 images...
[val/mtga] Cropping 112 images...
[val/mtgl] Cropping 68 images...
[val/sol] Cropping 87 images...

```

Done!

Full images in: data_aug_varsize\full_images

Cropped images in: data_aug_varsize\cropped_images